

Fundamentals of Computers and Computational Thinking

Course Code: CS101 Credits: 4 (Lecture: 3, Practical: 2) Total Hours: 75 Hrs

MODULE IV: Computational Thinking, Logic and Algorithms

Comprehensive Academic Notes — Units 16 through 22 (Duration: 13 Hours, Weightage: 20 Marks)

Unit 16: Computer Science as a Discipline — Scope & Grand Challenges

CONCEPTUAL DEFINITION: COMPUTER SCIENCE

Computer Science is the systematic study of algorithmic processes, computational machines, information structures, and the theoretical foundations of automation, encompassing software design, hardware architecture, complexity analysis, and mathematical modeling of computation.

Computer Science is not merely the study of computer hardware or programming syntax; as Edsger W. Dijkstra famously noted, "*Computer science is no more about computers than astronomy is about telescopes.*" It represents the formal science of problem-solving, information transformation, and the limits of automated computability. The discipline investigates three foundational questions: (1) *What can be computed?* (Computability Theory), (2) *How efficiently can it be computed?* (Computational Complexity), and (3) *How can practical physical and logical architectures be engineered to perform these computations reliably?* (Computer Systems and Software Engineering).

16.1 Sub-disciplines of Computer Science

- **Theoretical Computer Science:** Automata Theory, Formal Languages, Turing Machines, Computability, and Computational Complexity Classes (*P* vs *NP*, *NP-Complete*). Investigates fundamental questions regarding what can be solved within finite polynomial time versus what is undecidable (e.g., Turing's Halting Problem).
- **Computer Systems and Architecture:** Microprocessor Microarchitectures, Multi-core Symmetric Multiprocessing (SMP), Instruction-Level Parallelism (ILP), Distributed Systems, Operating Systems, Computer Networks, and Real-Time Systems.
- **Software Engineering:** Methodologies, design patterns, and formal verification frameworks for designing, architecting, implementing, testing, and maintaining large-scale, robust, fault-tolerant software systems.
- **Artificial Intelligence and Machine Learning:** Artificial Neural Networks, Deep Learning, Natural Language Processing, Computer Vision, Robotics, Knowledge Representation, and Autonomous Agent systems.
- **Data Science and Cryptography:** Big Data Analytics, Database Management Engines, Relational and NoSQL storage models, Cryptographic Protocols, Information Security, and Post-Quantum Cryptography.

16.2 Mathematical Foundations: Automata & The Turing Machine

In 1936, Alan Turing introduced the **Turing Machine (TM)**, a mathematical model of computation that defines the theoretical limits of automated computation. A Turing machine consists of an infinite memory tape divided into discrete cells, a read/write tape head that can scan, modify, and move left or right across tape cells, and a finite state control unit.

Formally, a deterministic Turing machine is defined as a 7-tuple:

$$M = \langle Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject} \rangle$$

where:

- Q is a finite non-empty set of internal machine states.
- Σ is the finite input alphabet not containing the blank symbol $\sqcup$.
- Γ is the finite tape alphabet ($\Sigma \subset \Gamma$ and $\sqcup \in \Gamma$).
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function mapping the current state and scanned tape symbol to a next state, a replacement symbol to write on the tape, and a directional tape head movement (Left or Right).
- $q_0 \in Q$ is the initial start state.
- $q_{accept} \in Q$ is the accepting halting state.
- $q_{reject} \in Q$ is the rejecting halting state ($q_{accept} \neq q_{reject}$).

The **Church-Turing Thesis** posits that any algorithmic calculation that can be naturally performed by a human mathematician following a deterministic effective procedure can be simulated by a Universal Turing Machine. This establishes the theoretical equivalence between Turing Machines, Lambda Calculus, and modern digital electronic computers.

16.3 Grand Challenges & Turing Award Milestones

The discipline is defined by historic milestones recognized by the ACM A.M. Turing Award (the "Nobel Prize of Computing"):

- **Alan Turing (1936):** Universal Turing Machine and Computability foundations. Proved the undecidability of the Halting Problem.
- **John von Neumann (1945):** Stored-program computer architecture unifying code and data in common addressable memory.
- **Claude Shannon (1948):** Mathematical Theory of Communication and Information Entropy ($H = -\sum p_i \log_2 p_i$).
- **Donald Knuth (1974):** Formalization of Algorithm Analysis, asymptotic notations, and *The Art of Computer Programming*.
- **Stephen Cook (1971) & Richard Karp (1972):** Foundations of **NP**-Completeness, Cook-Levin Theorem, and the 21 Karp reductions.

THEORETICAL PROOF: TURING'S HALTING PROBLEM IS UNDECIDABLE

Theorem: There exists no general algorithm that can determine whether an arbitrary program P will eventually halt (terminate) or run forever on an arbitrary input I .

Proof by Contradiction (Diagonalization):

1. Assume such a halting decider exists: a program $\text{Halt}(P, I)$ that returns `True` if $P(I)$ halts and `False` if $P(I)$ loops forever.
2. Construct an adversary program $\text{TuringOpposite}(P)$:

```
def TuringOpposite(P):  
    if Halt(P, P) == True:  
        while True: pass # Loop infinitely  
    else:  
        return True      # Halt immediately
```

3. What happens when we execute $\text{TuringOpposite}(\text{TuringOpposite})$?
 - If $\text{Halt}(\text{TuringOpposite}, \text{TuringOpposite})$ returns `True` (halts), TuringOpposite enters an infinite loop (does not halt).
 - If $\text{Halt}(\text{TuringOpposite}, \text{TuringOpposite})$ returns `False` (loops forever), TuringOpposite returns immediately (halts).
4. In both cases, we reach a direct logical contradiction ($P \iff \neg P$).

Conclusion: The initial assumption that a general halting decider exists is false. The Halting Problem is undecidable.

16.4 Complexity Classes: P , NP , NP -Complete, and NP -Hard

- **Class P (Polynomial Time):** The set of all decision problems that can be solved by a deterministic Turing machine in polynomial time $O(n^k)$ for some constant k (e.g., Shortest Path via Dijkstra, Sorting, Linear Search).
- **Class NP (Nondeterministic Polynomial Time):** The set of decision problems for which a candidate solution can be *verified* in polynomial time by a deterministic machine, or solved in polynomial time by a nondeterministic machine (e.g., Traveling Salesperson Problem, Boolean Satisfiability SAT, Subset Sum).
- **Class NP -Complete (NPC):** A problem X is NP -Complete if $X \in NP$ and every problem in NP can be polynomial-time reduced to X ($Y \leq_p X \forall Y \in NP$). If any single NP -Complete problem is solved in polynomial time, then $P = NP$.
- **Class NP -Hard:** Problems at least as hard as the hardest problems in NP , but not necessarily in NP (e.g., the Halting Problem, optimization version of TSP).
- **The P versus NP Millennium Problem:** The fundamental question asks: $P \stackrel{?}{=} NP$. If $P = NP$, then every problem whose solution can be quickly verified could also be quickly solved, fundamentally revolutionizing cryptography, artificial intelligence, and mathematics.
- **Post-Quantum Cryptography & Quantum Supremacy:** Shor's Algorithm solves prime factorization and discrete logarithms in polynomial time $O((\log N)^3)$ on a quantum computer, breaking RSA and ECC cryptography. The grand challenge is transitioning global cybersecurity to Lattice-based Post-Quantum Cryptography (e.g., CRYSTALS-Kyber and Dilithium).

Unit 17: Problem Solving Fundamentals & Software Development Lifecycle

CONCEPTUAL DEFINITION: PROBLEM SOLVING IN COMPUTING

Problem Solving in computer science is the systematic, structured cognitive process of analyzing an ill-defined real-world situation, identifying precise input constraints and expected output criteria, and formulating an executable sequence of deterministic operations that transforms inputs into valid solutions.

17.1 Polya's Four-Step Problem-Solving Framework

Renowned mathematician George Polya formulated a universal 4-step framework for mathematical and computational problem solving:

- 1. Understand the Problem:** Identify the unknowns, data inputs, operational conditions, and edge-case boundary constraints. Restate the problem in formal terminology, identifying preconditions (P_{pre}) and postconditions (P_{post}).
 - *Example:* Sorting an array requires knowing the data type, whether duplicate elements exist, memory constraints ($O(1)$ auxiliary space vs $O(N)$), and whether stability is required.
- 2. Devise a Plan:** Find connections between data and unknowns. Look for familiar patterns, decompose into smaller sub-problems, consider related algorithms (Divide-and-Conquer, Greedy, Dynamic Programming), or formulate a mathematical recurrence relation.
- 3. Carry Out the Plan:** Implement the algorithmic steps into clean, structured pseudocode or high-level code, checking each step for logical correctness, loop invariants, and boundary safety.
- 4. Look Back (Review & Refine):** Examine the resulting solution. Can you derive the result differently? Is the time and space complexity optimal? Does it handle extreme edge cases ($N = 0$, negative inputs, arithmetic overflow)?

17.2 The Input-Process-Output (IPO) Architecture

Every computational system, from a simple calculator to a multi-node cloud platform, fundamentally operates on the **Input-Process-Output (IPO)** model:

- **Input Phase (I):** Acquisition and validation of raw data from external peripherals (keyboard, sensors, network sockets, databases). Requires strict data validation, type checking, range assertion, and sanitization to prevent injection vulnerabilities.
- **Processing Phase (P):** Execution of deterministic algorithmic transformations, mathematical calculations, logical comparisons, data structure manipulations, and state transitions within CPU registers and RAM.
- **Output Phase (O):** Presentation or transmission of structured results to target destinations (monitors, disk storage, actuators, external APIs).

17.3 The 7-Stage Software Development Lifecycle (SDLC)

- 1. Problem Definition & Requirements Analysis:** Identifying stakeholder needs, specifying functional and non-functional requirements (SRS document), and establishing formal problem boundaries.
- 2. System & Algorithmic Design:** Designing data structures, algorithm steps, modular architecture, and creating flowcharts, pseudocode specifications, and ER diagrams.
- 3. Implementation / Coding:** Translating algorithmic blueprints into high-level programming language code (C, Python, Java) adhering to best practices and coding standards.
- 4. Verification, Formal Proofs & Testing:**
 - **Hoare Logic & Program Verification Triples:** Formal programs are verified using Hoare triples $\{P\} C \{Q\}$, where P is the precondition asserting state before command C executes, and Q is the postcondition guaranteed upon termination.
 - **Loop Invariant Verification:** Proving algorithm correctness requires a loop invariant assertion I that satisfies three properties:
 1. *Initialization:* I is true prior to the first iteration of the loop.
 2. *Maintenance:* If I is true before an iteration, it remains true before the next iteration.
 3. *Termination:* When the loop terminates, the invariant I combined with the loop termination condition proves the algorithm's postcondition.
 - *Black-Box Testing Techniques:* Testing functional behavior without internal code knowledge:
 - **Equivalence Class Partitioning (ECP):** Divides the input domain into valid and invalid equivalence classes, selecting representative values from each partition.
 - **Boundary Value Analysis (BVA):** Selects test inputs at the extreme boundary edges: **Min**, **Min + 1**, **Nominal**, **Max - 1**, **Max**.
 - *White-Box Testing & Cyclomatic Complexity:* Testing internal code structure, statement coverage, branch coverage, and path coverage. McCabe's Cyclomatic Complexity:

$$V(G) = E - N + 2P$$

where E is the number of control flow edges, N is the number of nodes, and P is the number of connected components. $V(G)$ defines the upper bound on the number of linearly independent execution paths requiring dedicated test cases.

- 5. **Deployment:** Releasing the verified binary executable or system into staging and production operational environments.
- 6. **Maintenance & Evolution:** Patching newly discovered defects (Corrective), optimizing performance (Perfective), adapting to new OS environments (Adaptive), and adding new features (Preventive).

Comparative Matrix: Software Engineering Methodologies

Methodology	Workflow Structure	Flexibility to Change	Ideal Project Type
Waterfall Model	Strict linear-sequential phases	Very Low (Rigid scope)	Mission-critical systems with unchanging specs (Avionics, Medical)
V-Model (Verification/Validation)	Paired development and testing stages	Low	Safety-critical embedded systems requiring rigorous test plans
Agile / Scrum	Iterative, incremental 2-4 week Sprints	Very High (Continuous feedback)	Commercial web apps, dynamic enterprise software, SaaS platforms

Unit 18: Four Pillars of Computational Thinking

CONCEPTUAL DEFINITION: COMPUTATIONAL THINKING

Computational Thinking (CT) is a formal thought process involved in formulating problems and their solutions in such a way that the solutions are represented in a form that can be effectively carried out by an information-processing agent (human or computer). Coined and popularized by Professor Jeannette Wing (2006).

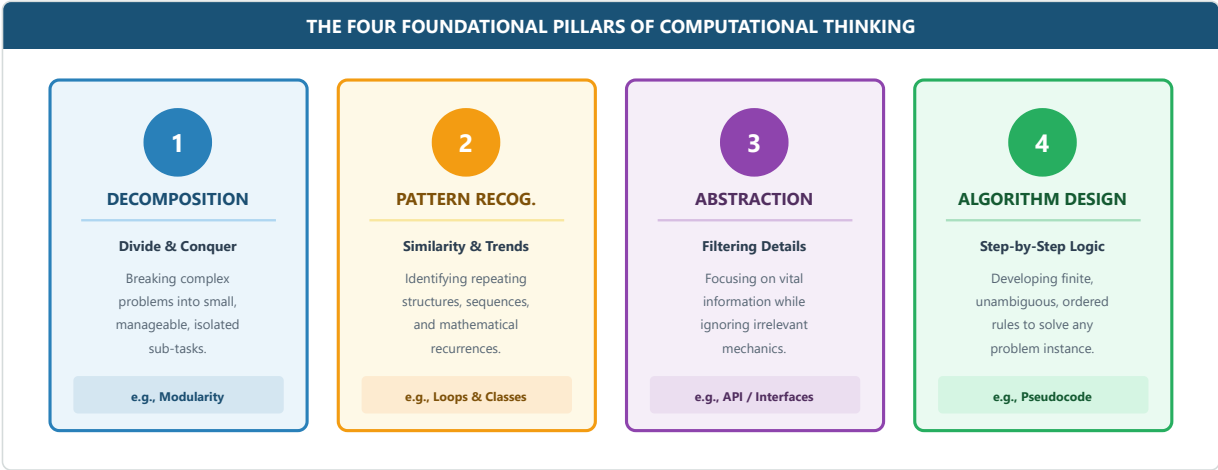


Figure 4.1: The Four Pillars of Computational Thinking.

18.1 Deep Dive into the Four Pillars with Real-World Case Studies

- 1. **Decomposition:** The analytical process of breaking down a large, complex, monolithic problem into smaller, self-contained, easily solvable sub-problems.
 - Case Study 1 (Autonomous Vehicle Navigation): A self-driving car system is too complex for a single program. Decomposition divides the system into: (a) Computer Vision & Sensor Fusion (processing camera, LIDAR, and radar streams), (b) Localization (GPS and IMU coordinate tracking), (c) Motion & Path Planning (Dijkstra / A* trajectory calculations), and (d) Actuation Control (CAN-bus steering, acceleration, and braking).
 - Case Study 2 (Compiler Engineering): Compilers decompose high-level programming language translation into distinct sequential phases: Lexical Analysis (Scanning source tokens), Syntax Analysis (Context-Free Grammar Parsing into Abstract Syntax Trees), Semantic Analysis (Type checking and scope validation), Intermediate Code Generation (Three-Address Code), Machine-Independent Optimization (Loop unrolling, dead code elimination), and Target Code Generation (Assembly / Machine Instructions).

2. **Pattern Recognition:** Observing and analyzing similarities, trends, periodic cycles, and recurring patterns within decomposed sub-problems to apply previously verified solutions.
- *Case Study 1 (Dynamic Programming & Bioinformatics):* In computing the global sequence alignment between two DNA strands (e.g., Needleman-Wunsch algorithm), recognizing that optimal alignment of prefix sequences $S_1[1..i]$ and $S_2[1..j]$ depends only on three previously computed adjacent states:
- $$M(i, j) = \max \begin{cases} M(i-1, j-1) + \text{Score}(S_1[i], S_2[j]) \\ M(i-1, j) + \text{GapPenalty} \\ M(i, j-1) + \text{GapPenalty} \end{cases}$$
- allows $O(mn)$ dynamic programming tabulation instead of $O(2^{m+n})$ exponential recursion.
- *Case Study 2 (Natural Language Processing):* Recognizing byte-pair encoding (BPE) subword patterns across vocabulary allows LLMs to tokenize arbitrary sentences into numerical embedding vectors efficiently.
3. **Abstraction:** The process of identifying the essential, high-level features of an object or process while filtering out and ignoring non-essential, low-level implementation details.
- *Case Study (GPS Map Routing):* A city road network is represented as an abstract mathematical Graph $G = (V, E)$ where road intersections are vertices (V) and streets are weighted edges (E). Irrelevant physical details (street tree counts, sidewalk material, building paint colors) are abstracted away, enabling graph algorithms (Dijkstra, Bellman-Ford) to find shortest routes efficiently.
 - *Case Study (Abstract Data Types):* Abstracting a `Queue` interface as `enqueue()`, `dequeue()`, and `isEmpty()` hides whether the underlying implementation uses a circular array or a doubly-linked list.
4. **Algorithm Design:** Formulating an ordered, deterministic sequence of unambiguous step-by-step instructions that can be mechanically executed to produce the correct solution for every valid input instance.

Unit 19: Logical Reasoning & Propositional Logic

19.1 Modes of Logical Reasoning

- **Deductive Reasoning (Top-Down Logic):** Derives guaranteed, logically certain conclusions from established true general premises.

Premise 1: All humans are mortal. Premise 2: Socrates is human. $\implies$ **Conclusion: Socrates is mortal.**

In programming, deductive reasoning guarantees that if an algorithm's preconditions and invariants hold, its postconditions must hold.

 - **Modus Ponens (Affirming the Antecedent):** If $P \rightarrow Q$ is True and P is True, then Q is True ($[(P \rightarrow Q) \wedge P] \implies Q$).
 - **Modus Tollens (Denying the Consequent):** If $P \rightarrow Q$ is True and Q is False ($\neg Q$), then P is False ($\neg P$) ($[(P \rightarrow Q) \wedge \neg Q] \implies \neg P$).
- **Inductive Reasoning (Bottom-Up Logic):** Formulates broad general hypotheses and probabilistic rules from specific empirical observations.

Observation: The sun rose in the east today, yesterday, and every recorded day. $\implies$ **Hypothesis: The sun always rises in the east.**

Serves as the foundation of Machine Learning and Statistical Artificial Intelligence.

 - **Abductive Reasoning:** Starts with an incomplete set of observations and proceeds to the most likely or simplest explanation (diagnostic debugging).

19.2 Propositional Logic & Truth Tables

A **Proposition** is a declarative statement that is either strictly **True (T/1)** or strictly **False (F/0)**, but never both simultaneously.

Fundamental Boolean Operators and Truth Table

P	Q	NOT $\neg P$	AND $P \wedge Q$	OR $P \vee Q$	XOR $P \oplus Q$	Implication $P \rightarrow Q$	Biconditional $P \leftrightarrow Q$
0	0	1	0	0	0	1	1
0	1	1	0	1	1	1	0
1	0	0	0	1	1	0	0
1	1	0	1	1	0	1	1

Fundamental Laws of Boolean Algebra:

- **De Morgan's Laws:**

$$\neg(P \wedge Q) \iff \neg P \vee \neg Q$$

$$\neg(P \vee Q) \iff \neg P \wedge \neg Q$$

Algebraic Proof of $\overline{X \cdot Y} = \bar{X} + \bar{Y}$ using Huntington Postulates:

1. Show $(X \cdot Y) + (\bar{X} + \bar{Y}) = 1$:

$$(X \cdot Y) + (\bar{X} + \bar{Y}) = ((X \cdot Y) + \bar{X}) + \bar{Y} = (X + \bar{X})(Y + \bar{X}) + \bar{Y} = 1 \cdot (Y + \bar{X}) + \bar{Y} = (Y + \bar{Y}) + \bar{X} = 1 + \bar{X} = 1$$

2. Show $(X \cdot Y) \cdot (\bar{X} + \bar{Y}) = 0$:

$$(X \cdot Y) \cdot \bar{X} + (X \cdot Y) \cdot \bar{Y} = (X \cdot \bar{X}) \cdot Y + X \cdot (Y \cdot \bar{Y}) = 0 \cdot Y + X \cdot 0 = 0 + 0 = 0$$

Since $\bar{X} + \bar{Y}$ satisfies both complementation axioms with respect to $(X \cdot Y)$, by uniqueness of complements, $\overline{X \cdot Y} = \bar{X} + \bar{Y}$.

- **Distributive Laws:**

$$P \wedge (Q \vee R) \iff (P \wedge Q) \vee (P \wedge R)$$

$$P \vee (Q \wedge R) \iff (P \vee Q) \wedge (P \vee R)$$

- **Absorption Laws:**

$$P \vee (P \wedge Q) \iff P, \quad P \wedge (P \vee Q) \iff P$$

- **Consensus Theorem:**

$$XY \vee \bar{X}Z \vee YZ \iff XY \vee \bar{X}Z$$

Proof: $XY + \bar{X}Z + YZ(X + \bar{X}) = XY + \bar{X}Z + XYZ + \bar{X}YZ = XY(1 + Z) + \bar{X}Z(1 + Y) = XY(1) + \bar{X}Z(1) = XY + \bar{X}Z$.

19.3 Karnaugh Maps (K-Maps) for Boolean Minimization

A **Karnaugh Map (K-Map)** is a graphical visual method for minimizing Boolean expressions into optimal Sum-of-Products (SOP) or Product-of-Sums (POS) forms without algebraic trial and error. Cells are arranged in a 2D grid where adjacent cells differ by exactly one binary bit (using **Gray Code ordering: 00, 01, 11, 10**). Groups of 2^k adjacent ones (**1, 2, 4, 8, 16**) are circled to eliminate redundant Boolean literals.

WORKED EXAMPLE 19.1: 3-VARIABLE K-MAP MINIMIZATION

Minimize the Boolean function $F(A, B, C) = \sum m(1, 3, 5, 7) + \sum m(4, 5)$.

Minterm Expansion:

$$F = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + ABC + A\bar{B}\bar{C}$$

K-Map Grid (Rows: $A \in \{0, 1\}$, Columns: $BC \in \{00, 01, 11, 10\}$):

- Row 0 ($A = 0$): Cell 01 (m_1) = 1, Cell 11 (m_3) = 1.
- Row 1 ($A = 1$): Cell 00 (m_4) = 1, Cell 01 (m_5) = 1, Cell 11 (m_7) = 1.

Grouping:

1. **Quad Group:** m_1, m_3, m_5, m_7 forms a 2×2 quad covering Column $C = 1$. Common literal = C .
2. **Pair Group:** m_4, m_5 forms a horizontal pair in Row 1 ($A = 1, B = 0$). Common literal = $A\bar{B}$.

Minimal Sum-of-Products (SOP) Expression: $F(A, B, C) = C + A\bar{B}$.

WORKED EXAMPLE 19.2: 4-VARIABLE K-MAP WITH DON'T CARE CONDITIONS

Minimize $F(A, B, C, D) = \sum m(0, 2, 8, 10, 14) + d(5, 15)$.

K-Map Grid (Rows: $AB \in \{00, 01, 11, 10\}$, Columns: $CD \in \{00, 01, 11, 10\}$):

- Row 00: $m_0(0000) = 1, m_2(0010) = 1$
- Row 01: $d_5(0101) = X$
- Row 11: $m_{14}(1110) = 1, d_{15}(1111) = X$
- Row 10: $m_8(1000) = 1, m_{10}(1010) = 1$

Optimal Grouping:

1. **Corner Quad:** Cells $(0, 2, 8, 10)$ at the four corners of the map wrap around vertically and horizontally. Common literals: $B = 0, D = 0 \Rightarrow \bar{B}\bar{D}$.
2. **Pair Group:** Cells $14, 15$ in Row 11 ($AB = 11, C = 1$). Common literals: ABC .

Minimal SOP Expression: $F(A, B, C, D) = \bar{B}\bar{D} + ABC$.

Unit 20: Algorithm Design & Complexity Analysis

CONCEPTUAL DEFINITION: ALGORITHM

An **Algorithm** is a finite, well-defined sequence of unambiguous, computer-implementable instructions designed to solve a specific class of problems or perform a computation.

20.1 Formal Characteristics of an Algorithm (Donald Knuth)

1. **Input:** An algorithm must have zero or more well-defined externally supplied quantities.
2. **Output:** An algorithm must produce at least one well-defined result related to the inputs.
3. **Definiteness (Unambiguity):** Every single step must be clear, precise, and have only one possible interpretation.
4. **Finiteness:** The algorithm must terminate after a finite number of computational steps for all valid inputs.
5. **Effectiveness:** Every operation must be sufficiently basic that it can in principle be executed exactly in finite time using pencil and paper.

20.2 Asymptotic Notations and Complexity Analysis

Algorithm efficiency is evaluated mathematically through asymptotic analysis of time and space complexity as input size $n \rightarrow \infty$:

- **Big-O Notation (O — Asymptotic Upper Bound):** Defines worst-case performance:

$$f(n) = O(g(n)) \iff \exists c > 0, n_0 \geq 1 \text{ such that } 0 \leq f(n) \leq c \cdot g(n) \forall n \geq n_0$$

- **Big-Omega Notation (Ω — Asymptotic Lower Bound):** Defines best-case performance:

$$f(n) = \Omega(g(n)) \iff \exists c > 0, n_0 \geq 1 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \forall n \geq n_0$$

- **Big-Theta Notation (Θ — Asymptotically Tight Bound):** Defines exact growth rate:

$$f(n) = \Theta(g(n)) \iff \exists c_1, c_2 > 0, n_0 \geq 1 \text{ such that } c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0$$

- **Little-o Notation (o — Strict Upper Bound):**

$$f(n) = o(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

- **Little-omega Notation (ω — Strict Lower Bound):**

$$f(n) = \omega(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

THE MASTER THEOREM FOR DIVIDE-AND-CONQUER RECURRENCES

For divide-and-conquer recurrences of the form $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, where $a \geq 1, b > 1$:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$. (Tree leaves dominate).
2. If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for $k \geq 0$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$. (e.g., Merge Sort: $T(n) = 2T(n/2) + \Theta(n) \implies a = 2, b = 2, \log_2 2 = 1 \implies \Theta(n \log n)$).
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and regularity condition $af(n/b) \leq cf(n)$ holds for $c < 1$, then $T(n) = \Theta(f(n))$. (Root work dominates).

MATHEMATICAL DERIVATION: MERGE SORT RECURRENCE VIA RECURSION TREE

Recurrence Relation: $T(n) = 2T(n/2) + c \cdot n$, with base case $T(1) = O(1)$.

Recursion Tree Structure:

- **Level 0 (Root):** 1 node of size $n \implies \text{Cost} = c \cdot n$.
- **Level 1:** 2 nodes of size $n/2 \implies \text{Cost} = 2 \times c(n/2) = c \cdot n$.
- **Level 2:** 4 nodes of size $n/4 \implies \text{Cost} = 4 \times c(n/4) = c \cdot n$.
- **Level k :** 2^k nodes of size $n/2^k \implies \text{Cost} = 2^k \times c(n/2^k) = c \cdot n$.

The recursion tree terminates when the subproblem size reaches $n/2^k = 1 \implies k = \log_2 n$.

Total Depth: $\log_2 n + 1$ levels.

Total Cost:

$$T(n) = \sum_{k=0}^{\log_2 n} c \cdot n = c \cdot n(\log_2 n + 1) = cn \log_2 n + cn = \Theta(n \log_2 n)$$

20.3 Classical Algorithmic Paradigms

- **Binary Search ($O(\log n)$):** In a sorted array, repeatedly halves the search interval. If target is less than midpoint, search left half; otherwise search right half. Recurrence: $T(n) = T(n/2) + O(1) \implies T(n) = O(\log n)$. Maximum comparisons: $\lfloor \log_2 n \rfloor + 1$.
- **Bubble Sort ($O(n^2)$):** Iteratively steps through the list, compares adjacent elements and swaps them if they are in the wrong order. Pass i bubbles the i -th largest element to its correct position. Total comparisons = $\sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2}$. Space complexity: $O(1)$ in-place.
- **Selection Sort ($O(n^2)$):** Finds the minimum element from the unsorted subarray and swaps it with the leftmost unsorted element. Number of comparisons is $\frac{n(n-1)}{2} = O(n^2)$ in all cases; swaps are strictly $O(n)$. Space complexity: $O(1)$ in-place.
- **Insertion Sort ($O(n^2)$ worst, $O(n)$ best):** Iteratively builds a sorted subarray by shifting larger elements one position right to insert the current key into its correct relative position. Ideal for nearly-sorted data. Space complexity: $O(1)$ in-place.
- **Merge Sort ($O(n \log n)$):** Divide-and-conquer algorithm dividing array into two halves, recursively sorting both, and merging them in linear $O(n)$ time using $O(n)$ auxiliary memory space.
- **Euclidean Algorithm for GCD ($O(\log(\min(a, b)))$):** Computes the greatest common divisor of two integers a and b ($a > b$) using the principle $\gcd(a, b) = \gcd(b, a \bmod b)$ until remainder is zero.

Unit 21: Flowcharts & Visual Algorithm Modeling

CONCEPTUAL DEFINITION: FLOWCHART

A **Flowchart** is a formalized diagrammatic representation of an algorithm displaying the sequential step-by-step workflow, data transformations, decision branching, and loop iterations using standardized geometric symbols connected by directional flowlines.

21.1 Standard Flowchart Symbols (ISO 5807 / RAPTOR Standard)

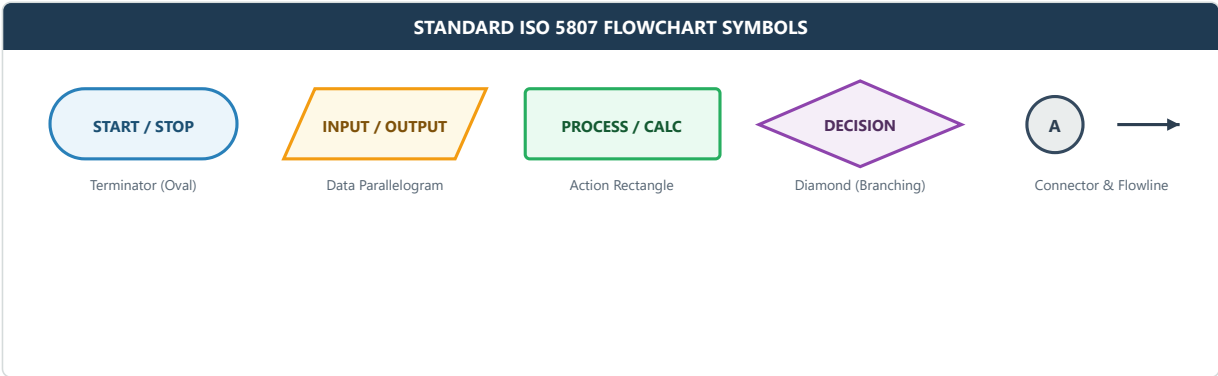


Figure 4.2: Standard Flowchart Graphical Elements for Algorithm Representation.

21.2 Structured Programming Control Constructs

The **Böhm-Jacopini Structured Programming Theorem (1966)** established that any computable algorithm or Turing-computable function can be expressed using only three basic control flow structures without resorting to arbitrary `GOTO` jumps:

- **Sequence Structure:** Linear progression of instructions executed strictly in the order they appear without branching.
- **Selection Structure (Conditional Branching):** Chooses between alternative execution paths based on a Boolean condition:
 - `IF-THEN` (Single alternative)
 - `IF-THEN-ELSE` (Dual alternative)
 - `SWITCH-CASE` (Multi-way branch evaluated via jump table)
- **Iteration Structure (Repetition / Loops):** Repeatedly executes a block of statements while a condition remains true:
 - *Pre-Test Loop* (`WHILE` loop): Condition tested before entering loop body; executes zero or more times.
 - *Post-Test Loop* (`DO-WHILE` loop): Condition tested after executing loop body; executes at least once.
 - *Count-Controlled Loop* (`FOR` loop): Uses initialization, test condition, and step increment variables.

21.3 The RAPTOR Visual Programming Environment

RAPTOR (Rapid Algorithmic Prototyping Tool for Ordered Reasoning) is a visual flowchart-based programming environment developed specifically to help novice computer science students design and execute algorithms without syntax errors. RAPTOR executes flowcharts step-by-step, highlighting active nodes and displaying dynamic variable watches in real time.

Comparative Analysis: Algorithm vs Flowchart vs Pseudocode

Dimension	Natural Language Algorithm	Graphical Flowchart	Structured Pseudocode
Format	Descriptive plain English text sentences	Standardized 2D geometric symbols (ISO 5807)	Language-independent structured text syntax
Visual Scannability	Low; requires reading full text paragraphs	Very High; branches and loops instantly visible	High; indentation reveals block nesting levels
Ease of Modification	High; simple text re-writing	Low; requires redrawing layout lines and shapes	Very High; direct keyboard edits
Translability to Code	Moderate; potential semantic ambiguities	High; maps directly to structured programming	Very High; almost 1-to-1 mapping with C/Python code

Unit 22: Practical Lab Implementations — Flowcharts & Algorithms

Comprehensive step-by-step algorithms, mathematical formulations, and trace tables for all 6 prescribed syllabus laboratory problems.

LAB PROBLEM 1: FIND THE LARGEST OF THREE NUMBERS (A, B, C)

Problem Statement: Read three distinct numerical values A, B, C from standard input and determine which number holds the maximum value.

Mathematical Formulation: $\text{Max}(A, B, C) = \max(A, \max(B, C))$.

```
Algorithm FindLargest(A, B, C):
1. START
2. INPUT A, B, C
3. IF (A >= B) THEN:
    IF (A >= C) THEN:
        DISPLAY "A is Largest"
    ELSE:
        DISPLAY "C is Largest"
    ELSE:
        IF (B >= C) THEN:
            DISPLAY "B is Largest"
        ELSE:
            DISPLAY "C is Largest"
4. STOP
```

Trace Table ($A = 45, B = 78, C = 23$):

Step	Condition Checked	Boolean Evaluation	Action Taken
1	Input $A = 45, B = 78, C = 23$	True	Variables assigned in memory
2	$A \geq B \implies 45 \geq 78$	False	Branch to Else ($B > A$)
3	$B \geq C \implies 78 \geq 23$	True	Execute: Display "B is Largest: 78"

LAB PROBLEM 2: CALCULATE SUM OF FIRST N NATURAL NUMBERS

Problem Statement: Compute the cumulative arithmetic sum $S = \sum_{i=1}^N i = 1 + 2 + 3 + \dots + N$ for a positive integer N .

Closed-Form Formula: $S = \frac{N(N+1)}{2}$. Time Complexity: Iterative $O(N)$, Closed-Form $O(1)$.

```
Algorithm SumOfNaturalNumbers(N):
1. START
2. INPUT N
3. INITIALIZE Sum = 0, Counter = 1
4. WHILE (Counter <= N) DO:
    Sum = Sum + Counter
    Counter = Counter + 1
5. DISPLAY "Sum is:", Sum
6. STOP
```

Trace Table for $N = 5$:

Iteration	Counter (i)	Condition ($i \leq 5$)	Sum Expression	Updated Sum
Initial	1	-	0	0
1	1	$1 \leq 5$ (True)	$0 + 1$	1
2	2	$2 \leq 5$ (True)	$1 + 2$	3
3	3	$3 \leq 5$ (True)	$3 + 3$	6
4	4	$4 \leq 5$ (True)	$6 + 4$	10
5	5	$5 \leq 5$ (True)	$10 + 5$	15
Exit	6	$6 \leq 5$ (False)	Loop Terminates	Final: 15

LAB PROBLEM 3: COMPUTE FACTORIAL OF A NUMBER (N!)

Mathematical Formulation: $N! = \prod_{i=1}^N i = 1 \times 2 \times \dots \times N$, with base case $0! = 1$.

```
Algorithm ComputeFactorial(N):
1. START
2. INPUT N
3. IF (N < 0) THEN:
    DISPLAY "Error: Negative numbers have no real factorial"
    GOTO Step 7
4. INITIALIZE Fact = 1, i = 1
5. WHILE (i <= N) DO:
    Fact = Fact * i
    i = i + 1
6. DISPLAY "Factorial is:", Fact
7. STOP
```

Trace Table for N = 4:

Step	Loop Variable (i)	Condition (i ≤ 4)	Calculation (Fact × i)	Updated Fact
Init	1	-	-	1
1	1	1 ≤ 4 (True)	1 × 1	1
2	2	2 ≤ 4 (True)	1 × 2	2
3	3	3 ≤ 4 (True)	2 × 3	6
4	4	4 ≤ 4 (True)	6 × 4	24
Exit	5	5 ≤ 4 (False)	Loop Terminated	Final: 24

LAB PROBLEM 4: GENERATE FIBONACCI SEQUENCE UP TO N TERMS

Recurrence Relation: $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$ for $n \geq 2$.

```
Algorithm GenerateFibonacci(N):
1. START
2. INPUT N
3. IF (N <= 0) THEN GOTO Step 8
4. INITIALIZE T1 = 0, T2 = 1, Count = 1
5. WHILE (Count <= N) DO:
    DISPLAY T1
    NextTerm = T1 + T2
    T1 = T2
    T2 = NextTerm
    Count = Count + 1
6. STOP
```

Trace Table for $N = 6$:

Term Number	Displayed Value (T_1)	NextTerm ($T_1 + T_2$)	New T_1	New T_2
1	0	$0 + 1 = 1$	1	1
2	1	$1 + 1 = 2$	1	2
3	1	$1 + 2 = 3$	2	3
4	2	$2 + 3 = 5$	3	5
5	3	$3 + 5 = 8$	5	8
6	5	$5 + 8 = 13$	8	13

LAB PROBLEM 5: CHECK WHETHER A NUMBER IS PRIME

Mathematical Definition: An integer $N > 1$ is prime if and only if it has no positive divisors other than 1 and itself. Optimization: Test trial division up to $\lfloor \sqrt{N} \rfloor$, because if $N = a \times b$, at least one factor must satisfy $a \leq \sqrt{N}$.

```
Algorithm CheckPrime(N):
1. START
2. INPUT N
3. IF (N <= 1) THEN:
    DISPLAY "N is neither prime nor composite"
    GOTO Step 8
4. INITIALIZE IsPrime = True, Divisor = 2
5. WHILE (Divisor * Divisor <= N) DO:
    IF (N MOD Divisor == 0) THEN:
        IsPrime = False
        BREAK
    Divisor = Divisor + 1
6. IF (IsPrime == True) THEN:
    DISPLAY "N is a Prime Number"
ELSE:
    DISPLAY "N is a Composite Number"
7. STOP
```

Trace Table for Test Case $N = 29$ ($\lfloor \sqrt{29} \rfloor = 5$):

Step	Divisor	$\text{Divisor}^2 \leq 29$	$29 \bmod \text{Divisor}$	IsPrime State	Loop Action
Init	2	-	-	True	Enter Loop
1	2	$4 \leq 29$ (True)	$29 \bmod 2 = 1 \neq 0$	True	Divisor $\rightarrow$ 3
2	3	$9 \leq 29$ (True)	$29 \bmod 3 = 2 \neq 0$	True	Divisor $\rightarrow$ 4
3	4	$16 \leq 29$ (True)	$29 \bmod 4 = 1 \neq 0$	True	Divisor $\rightarrow$ 5
4	5	$25 \leq 29$ (True)	$29 \bmod 5 = 4 \neq 0$	True	Divisor $\rightarrow$ 6
Exit	6	$36 \leq 29$ (False)	-	True	Terminate: 29 is Prime

Trace Table for Test Case $N = 35$ ($\lfloor \sqrt{35} \rfloor = 5$):

Step	Divisor	$\text{Divisor}^2 \leq 35$	$35 \bmod \text{Divisor}$	IsPrime State	Loop Action
1	2	$4 \leq 35$ (True)	$35 \bmod 2 = 1 \neq 0$	True	Divisor $\rightarrow$ 3
2	3	$9 \leq 35$ (True)	$35 \bmod 3 = 2 \neq 0$	True	Divisor $\rightarrow$ 4
3	4	$16 \leq 35$ (True)	$35 \bmod 4 = 3 \neq 0$	True	Divisor $\rightarrow$ 5
4	5	$25 \leq 35$ (True)	$35 \bmod 5 = 0$	False	Break: 35 is Composite

LAB PROBLEM 6: LINEAR SEARCH IN AN ARRAY

Problem Statement: Search for a target key element K in an unsorted array A of size N .

Complexity: Best Case $O(1)$ (target at index 0), Worst Case $O(N)$ (target at last index or absent), Space Complexity $O(1)$.

```
Algorithm LinearSearch(A, N, Key):
1. START
2. INPUT Array A, Size N, Search Key K
3. INITIALIZE Index = 0, Found = False
4. WHILE (Index < N) DO:
    IF (A[Index] == Key) THEN:
        DISPLAY "Element found at index:", Index
        Found = True
        BREAK
    Index = Index + 1
5. IF (Found == False) THEN:
    DISPLAY "Element not present in array"
6. STOP
```

Trace Table ($A = [42, 17, 89, 53, 26]$, $N = 5$, Key = 89):

Iteration	Index (i)	Condition ($i < 5$)	Array Element ($A[i]$)	Comparison ($A[i] == 89$)	Action
1	0	$0 < 5$ (True)	42	$42 == 89$ (False)	$i \rightarrow 1$
2	1	$1 < 5$ (True)	17	$17 == 89$ (False)	$i \rightarrow 2$
3	2	$2 < 5$ (True)	89	$89 == 89$ (True)	Found at Index 2; Break

Trace Table for Absent Key ($A = [42, 17, 89, 53, 26]$, $N = 5$, Key = 99):

Iteration	Index (i)	Condition ($i < 5$)	Array Element ($A[i]$)	Comparison ($A[i] == 99$)	Action
1	0	$0 < 5$ (True)	42	$42 == 99$ (False)	$i \rightarrow 1$
2	1	$1 < 5$ (True)	17	$17 == 99$ (False)	$i \rightarrow 2$
3	2	$2 < 5$ (True)	89	$89 == 99$ (False)	$i \rightarrow 3$
4	3	$3 < 5$ (True)	53	$53 == 99$ (False)	$i \rightarrow 4$
5	4	$4 < 5$ (True)	26	$26 == 99$ (False)	$i \rightarrow 5$
Exit	5	$5 < 5$ (False)	-	-	Display "Not present"